Progreso del curso · 32 capítulos, 9 partes

Parte 0
Introducción

1 2

Parte I
Fundamentos estructurales y de datos

3 4 5◇ 6 7 8 9

Parte II
Estimación flexible y primer despliegue

10 11 12 13 14 15

Parte III
Diseños de identificación con ML

16◇ 17◇ 18◇

Parte IV
Publicación y operación en producción

19 **20**

Parte V
Heterogeneidad, políticas y targeting

21 22

Parte VI
Explicación, equidad y decisión

23 24 25 26

Parte VII
Métodos avanzados

27◇ 28◇

Parte VIII
Estructura, monitoreo y frontera

29 30 31 32

■ previos · ■ actual · □ próximos · ◇ profundización electiva (no bloquea el camino ⋆)

20

CAPÍTULO 20

★ Camino crítico

CI/CD para modelos causales

Parte IV · Sesión S18

Plantilla B (ingeniería y producción)

IA Causal en Producción

Curso ECO-490 / ECON-540 · Edición 2026

Roadmap del capítulo

Objetivos de aprendizaje

1. Diseñar un pipeline con etapa de software y etapa estadística separadas.
2. Escribir una prueba de humo tipo placebo que bloquee publicación.
3. Situar el pipeline del curso dentro del ecosistema MLOps estándar.

★ Antes de clase

Lectura: Cap. 20 — CI/CD causal (@cap-cicd-causal)
Lab: L18
Entrega/hito: B2 entrega · Validar con grade_b2.py

Agenda

01 · Un CI genérico no detecta que un placebo dejó de dar cero

02 · CI genérico vs. CI con pruebas estadísticas integradas

03 · Decisión: dos etapas, software + estadística

04 · Workflow + prueba de humo placebo

05 · Dónde encaja MLflow/DVC/Great Expectations/Evidently

El problema de diseño

Decisión	El pipeline de CI debe correr pruebas de sanidad estadística, no solo pruebas de código.
Alternativas	CI genérico (solo tests de software) vs. CI con pruebas estadísticas causales integradas.
Trade-off clave	Tiempo de ejecución del pipeline vs. riesgo de publicar automáticamente un modelo roto.
Stack	GitHub Actions, pytest, Docker.
Riesgo principal	Que un cambio de código pase todos los tests de software y aun así rompa la validez causal.

Opciones y trade-offs

CI genérico
¿El código importa sin error? ¿El endpoint responde? Rápido, pero un cambio que introduce un mal control pasa sin ninguna advertencia.

CI con pruebas estadísticas
Placebo + chequeo de balance como parte del pipeline, con umbrales que bloquean la publicación automática. Más lento, requiere dataset de referencia fijo.

Decisión adoptada

Pipeline en dos etapas: software (segundos, cada push) y estadística (minutos, solo antes de publicar al Hub).

Evita que cada cambio menor dispare un reentrenamiento costoso, sin sacrificar la garantía de que nada se publica sin pasar la prueba de sanidad estadística.

Pipeline de CI/CD con gate estadístico



✗ bloquea publicar-hub si placebo o balance fallan

Workflow con dos etapas

```yaml
# .github/workflows/ci-cd.yml
name: CI/CD modelo causal
on: [push]

jobs:
  tests-software:          # segundos, en cada push
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with: { python-version: '3.11' }
      - run: pip install -r requirements.txt
      - run: pytest tests/test_software.py -v

  tests-estadisticos:      # minutos, solo antes de publicar al Hub
    needs: tests-software
    if: github.ref == 'refs/heads/main'
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: pip install -r requirements.txt
      - run: python scripts/validar_schema.py  # card vs causal-model-card-v1.schema.json (Ap. G.4)
      - run: pytest tests/test_placebo.py -v --tb=short
      - run: pytest tests/test_balance.py -v --tb=short

  publicar-hub:            # bloqueado si tests-estadisticos falla
    needs: tests-estadisticos
    if: github.ref == 'refs/heads/main'
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: pip install huggingface_hub
      - run: python scripts/publicar_hub.py
        env:
          HF_TOKEN: ${{ secrets.HF_TOKEN }}
```

Prueba de humo: placebo no significativo

```python
# tests/test_placebo.py
import joblib
import pandas as pd
import numpy as np

def test_placebo_no_significativo():
    modelo = joblib.load('model/dml_estimator.joblib')
    df = pd.read_parquet('data/referencia_ci.parquet')  # dataset fijo de referencia
    rng = np.random.default_rng(0)
    D_placebo = rng.permutation(df['educ'].values)  # tratamiento aleatorizado
    X = df[['exper', 'black', 'south', 'married']]
    efecto_placebo = modelo.effect(X).mean()
    ic_inf, ic_sup = modelo.effect_interval(X, alpha=0.05)
    assert ic_inf.mean() <= 0 <= ic_sup.mean(), (
        f'Placebo significativo: IC=({ic_inf.mean():.3f}, {ic_sup.mean():.3f})'
    )
```

02_monografia → 03_notebooks_caso_estudio / 04_notebooks_produccion_hf / 05_labs_sesion

.github/workflows/b1_smoke.yml y
course_practice_ci.yml (pytest Serie A, labs, humo B1-B4). Lab L18. Autograders:
grade_b1.py...grade_b4.py. Auditoría local: bash scripts/audit_repo.sh.

Recordatorio del caso

MLflow/W&B para experimentos, DVC para versionar el dataset de referencia, Great Expectations/Pandera para validar esquema, Evidently para drift.

Resultado numérico

Ninguna de estas herramientas sabe qué es un placebo, un chequeo de balance o un IC sobre un efecto — se detienen en integridad de datos y métricas predictivas.

Lectura de política · La etapa tests-estadisticos es específica del dominio causal y hay que escribirla; el resto del ecosistema no se reimplementa.

Nivel PCH	**Estimando / métrica principal**
Transversal (garantiza que L2 siga siendo válido tras cada cambio)	Placebo + balance como gate de publicación automática

Pregunta activa (1 min)

Un cambio legítimo en el mundo también puede romper un umbral estadístico — ¿por qué el pipeline no debe "arreglar el código hasta que pase" en ese caso?

Escalera PCH: L1 (asociación) ⊆ L2 (intervención) ⊆ L3 (contrafactual) — cada nivel exige supuestos estrictamente más fuertes que el anterior. Si solo predijéramos, ¿a quién trataríamos? Con el efecto, ¿cambia la decisión?

Señales de alerta y checklist operativo

- ☐ El pipeline solo corre pruebas de software; ningún job verifica placebo o balance.
- ☐ El dataset de referencia no está versionado — las pruebas dan resultados distintos en ejecuciones distintas.
- ☐ La publicación ocurre en el mismo job que las pruebas de software.
- ☐ Nadie revisa manualmente los resultados de las pruebas estadísticas cuando fallan.

Si dos o más señales están presentes, el sistema no está listo según los criterios de este curso.

Punto de quiebre · Aplicado

★ Tres criterios de diseño de la etapa estadística: dataset congelado y versionado, umbrales fijados antes (no ajustados al fallar), fallo estadístico ≠ fallo de software. ¿Cuál se viola con más frecuencia en la práctica?

⏱ 2-4 min · Votación mano alzada o Mentimeter

Discusión y ejercicio

Pregunta de discusión

¿Por qué separar pruebas de software (cada push) de pruebas estadísticas (solo antes de publicar) es mejor diseño que correr ambas en cada commit?

Ejercicio (5-10 min)

Complete el Notebook B2: implemente el workflow y las pruebas, verifique que remover un control clave bloquea el job publicar-hub.

Cierre del capítulo

Resumen

- Un CI genérico no detecta ruptura de validez causal.
- Dos etapas: software en cada push, estadística antes de publicar.
- El ecosistema MLOps estándar cubre integridad de datos y drift; la sanidad causal (placebo, balance) es responsabilidad del equipo.

Referencias clave

(1) Huyen (2022) / Kästner et al. (2024), CI/CD para ML

(2) Breck et al. (2017), ML Test Score

Próximo episodio · Cap. 23 — Explicación causal y equidad algorítmica (S19)

Gancho: Con el sistema desplegado, publicado y auditado automáticamente, la pregunta cambia de validez estadística a validez frente a personas: ¿el efecto se explica igual que se predice?